

Multimedia Computing

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University in Tashkent.

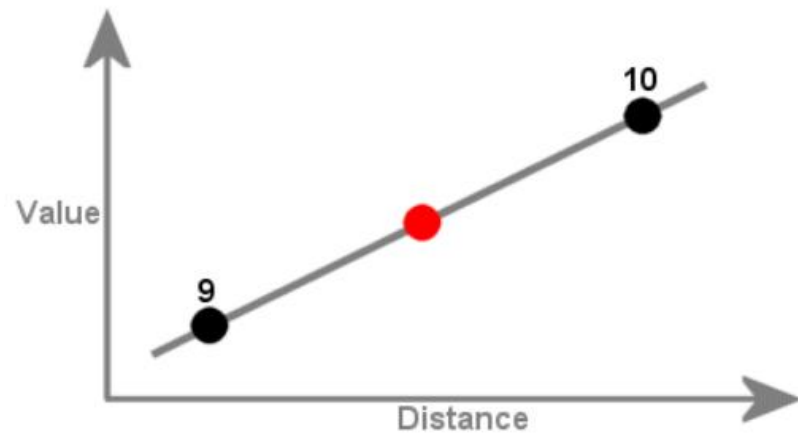
Email: minhaz.ahmed@gmail.com

Content

- ▶ Interpolation
- ▶ Nearest Neighbor interpolation
- ▶ Bilinear interpolation
- ▶ Cubic convolution interpolation
- ▶ Spline interpolation
- ▶ Image scaling
- ▶ Minification
- ▶ Rotation
- ▶ Translation
- ▶ Mirroring

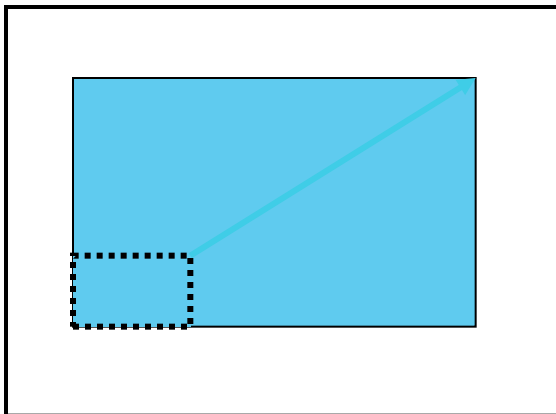
Interpolation

- ▶ The process of using known data values to estimate unknown data values.

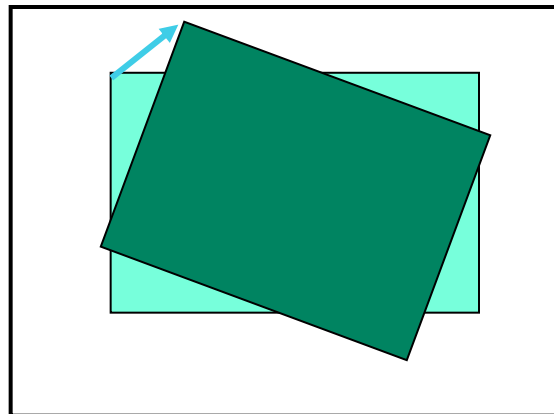


Geometric Processes

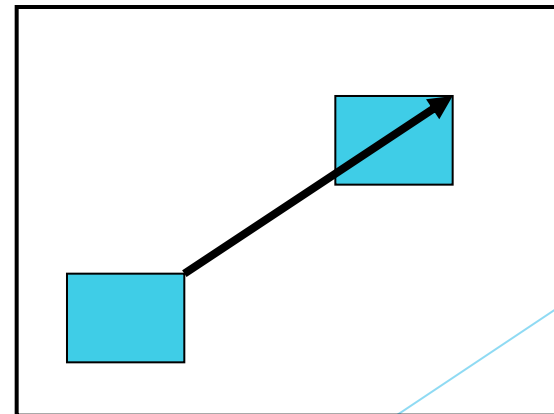
- ◆ Modify the arrangement of pixel based on some geometric transformation.
- ◆ Ideally the pixel values are not altered.
 - ◆ Newly pixel may be generated if the geometric process attempts to source a non-existing pixel => Interpolation
 - ◆ Interpolation is used by many geometric transformations.
- ◆ Basic geometric processes : **scaling, rotation and translation**



Scale



Rotation



Translation

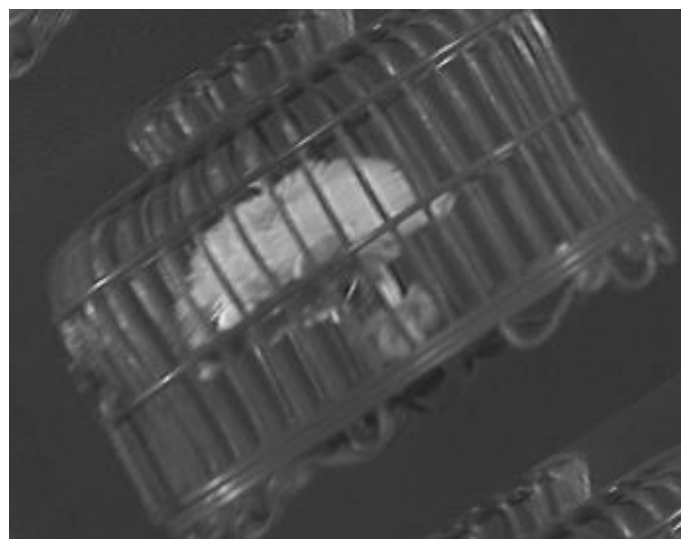
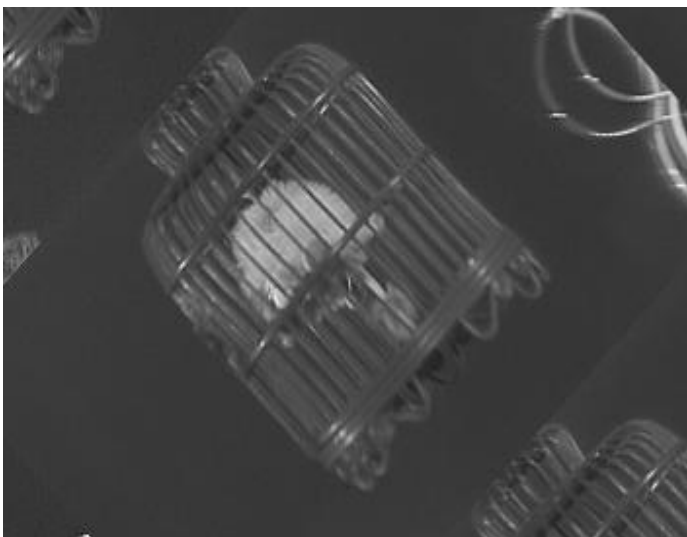
Geometric Processes

Original



**$S_x=2,$
 $S_y=2$**

$\theta = 45^\circ$



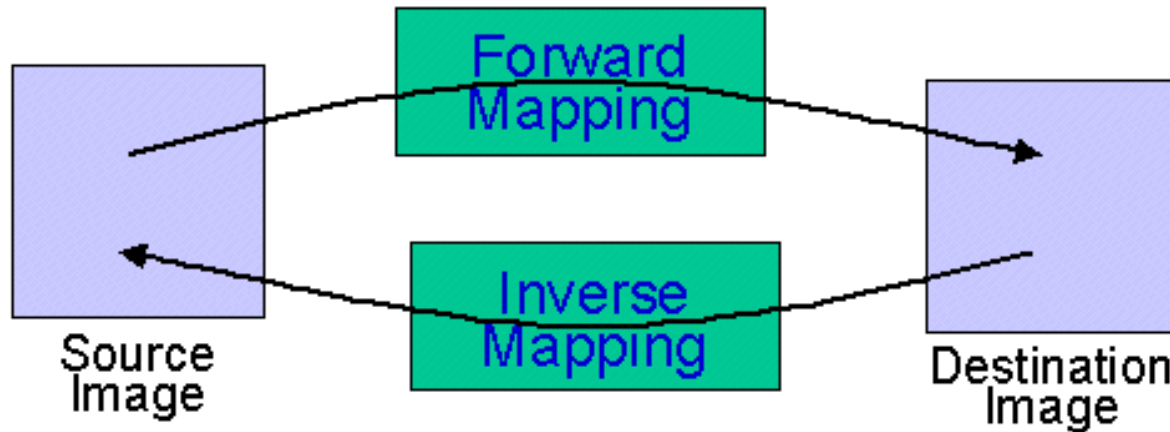
**$S_x=2,$
 $S_y=1,$
 $\theta = 30^\circ$**

Forward vs. Reverse Mapping

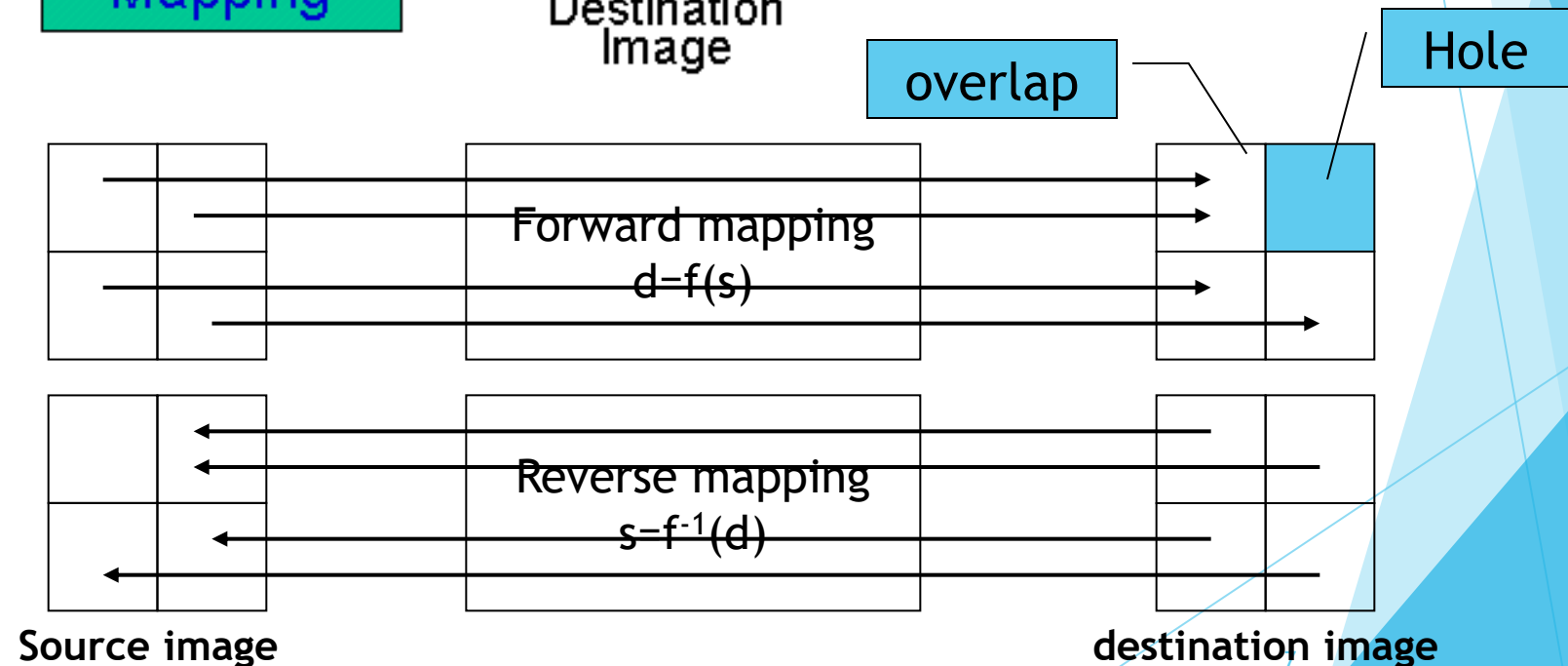
✚ Forward mapping

- ◆ Source image $\rightarrow$ forward mapping $\rightarrow$ destination image
 - ◆ Transferring the input image to the output via some x and y mapping function.
- ◆ Two problems with forward mapping function
- ◆ Holes :
 - ◆ Pixels that are undefined
 - ◆ $\Rightarrow$ destination image pixel has no corresponding source pixel
- ◆ Overlaps :
 - ◆ Two input pixels get mapped to the same output pixel.

Forward vs. Reverse Mapping



Any geometric transformation, including an affine transformation, can be implemented as forward or backward mapping. The forward mapping **iterates over each pixel of the input image, computes new coordinates for it, and copies its value to the new location.**



Forward vs. Reverse Mapping

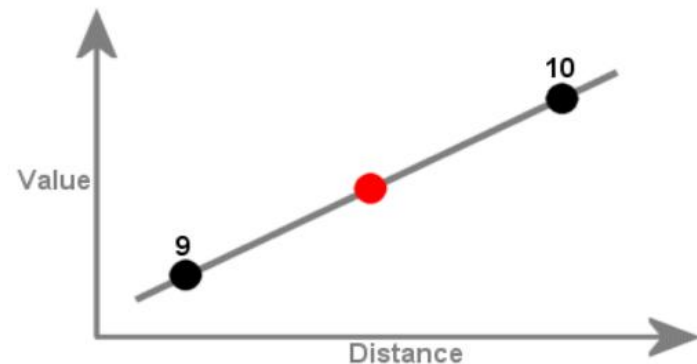
- ✚ Reverse mapping
 - ◆ Traverses the destination images
 - ◆ Calculate which pixel in the source image will be used to produce destination pixel via some inverse transformation.
 - ◆ Eliminate the problems of holes and overlaps.

Interpolation

- ▶ The process of generating values for addresses between pixels.
- ▶ Right interpolation function is application dependent.
 - ▶ Trade off between quality and processing time
 - ▶ More sophisticated algorithms -> improve image quality -> more complex interpolation -> more processing time.
- ◆ Ex. A transformation function: (use known data to estimate unknown location)

$$x_{source} = \frac{x_{dest}}{2} \quad y_{source} = \frac{y_{dest}}{2}$$

- ◆ $dest[0][0] \leftarrow source[0][0]$
- ◆ $dest[1][1] \leftarrow source[0.5][0.5]$



Interpolation

- ▶ Interpolation used for zooming , shrinking, rotating and geometric processing.
- ▶ Zooming requires two steps
 - ▶ The creation of new pixel locations
 - ▶ The assignment of gray levels to these new locations
- ▶ **Techniques for zooming**
 - ▶ Nearest Neighbor Interpolation
 - ▶ Bilinear interpolation
 - ▶ Bicubic interpolation

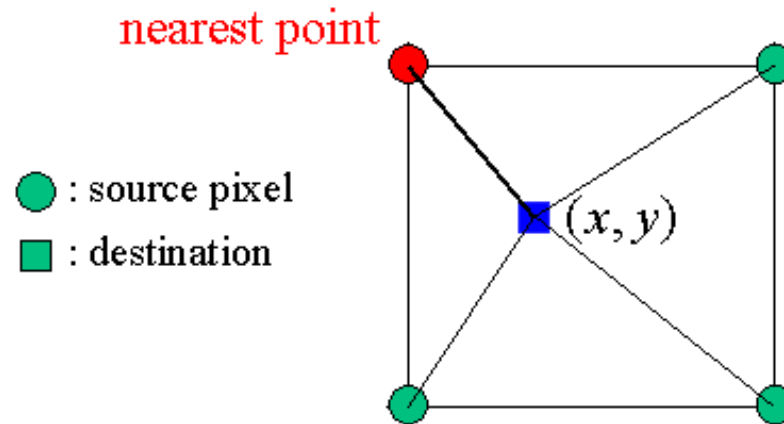
1. Nearest Neighbor Interpolation

✚ Nearest Neighbor Interpolation

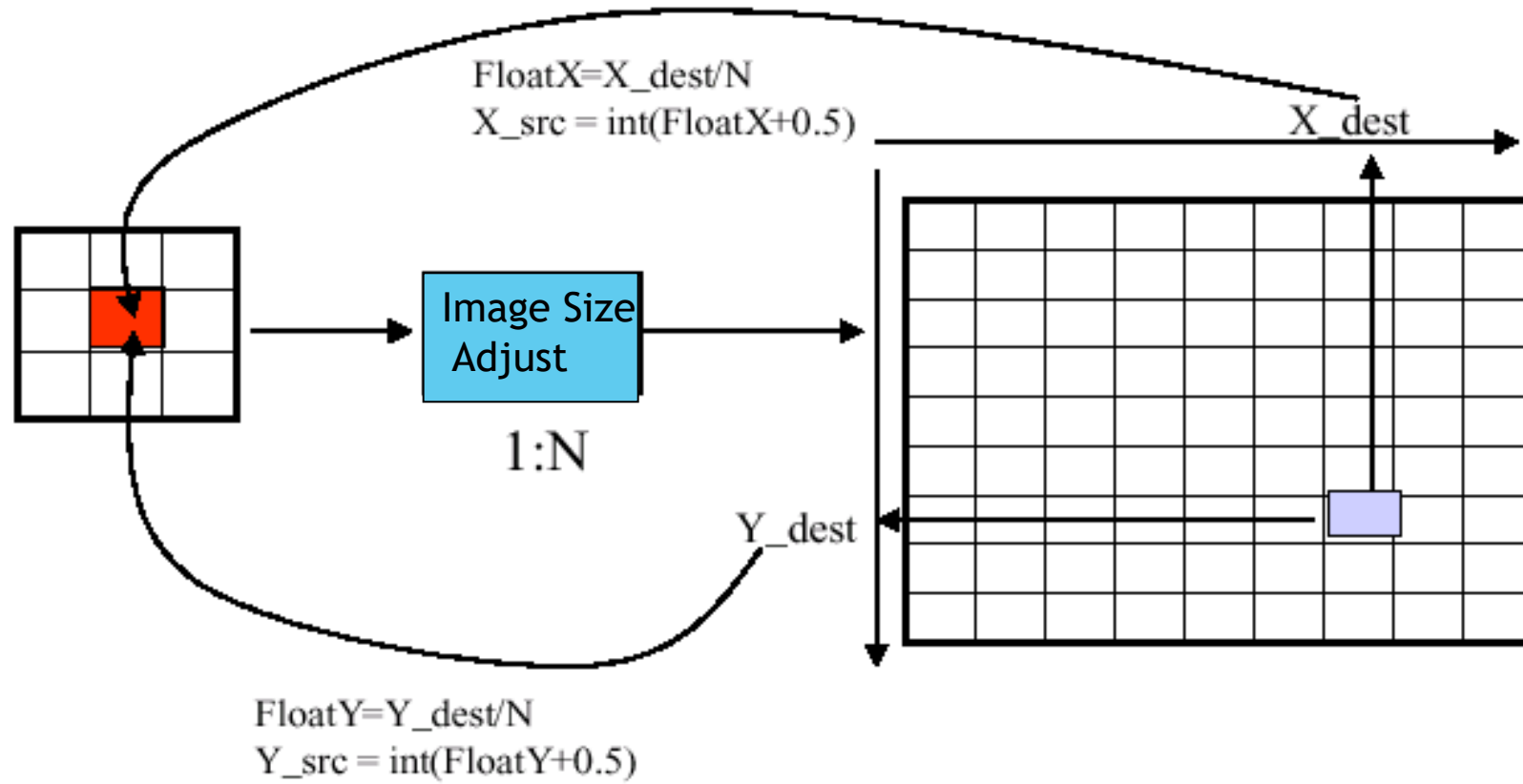
- ◆ (Basic idea) Assign the pixel closest to the newly generated address as output pixel.
 - ◆ The fractional address computed for the source pixel is rounded to the nearest valid pixel address.
- ◆ Fast, but yields greatly varying results.
 - ◆ The greater the number of output pixels tied to one input pixel, the worse the output looks.

➤ code

```
floatx = x_mapping_function(X_dest);  
floaty = y_mapping_function(Y_dest);  
X_source = (int)(floatX+0.5);  
Y_source = (int)(floatY+0.5);
```

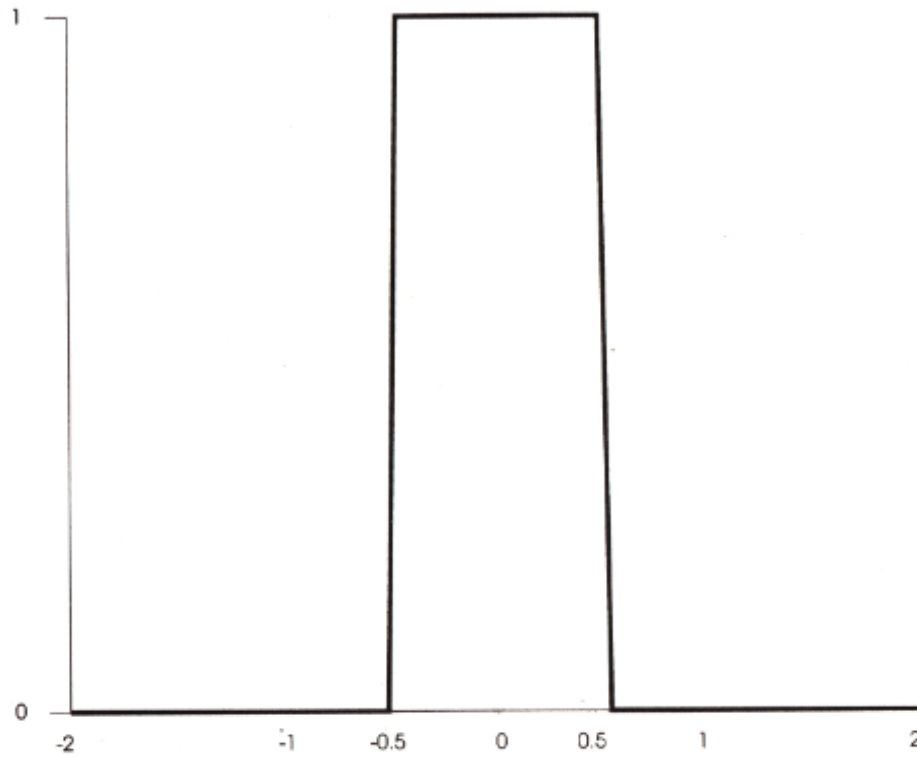


Nearest Neighbor Interpolation



Nearest Neighbor Interpolation

- ▶ Nearest neighbor interpolation function



Nearest Neighbor Interpolation

▶ **Process:**

- For each non-integer coordinate, find the nearest neighboring pixel.
- Assign the value of that nearest pixel to the new location.

➤ **Pros:**

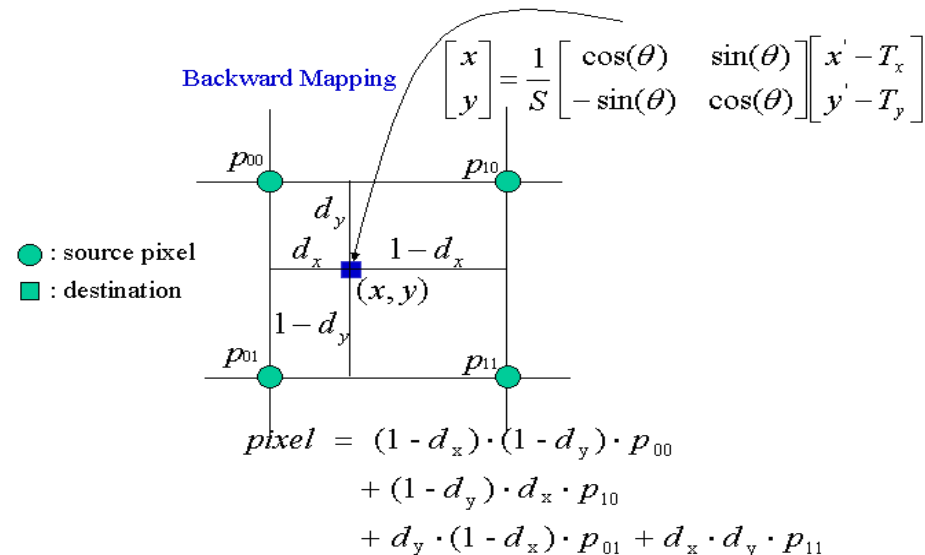
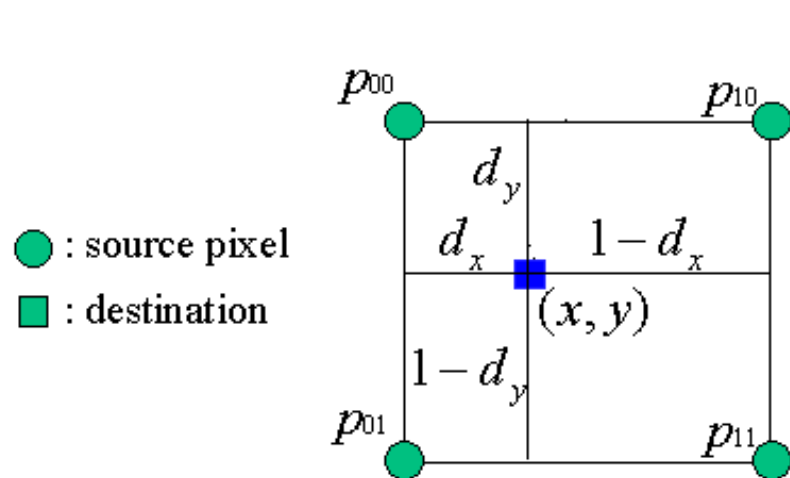
Simple and computationally inexpensive.

➤ **Cons:**

The resulting image can appear pixelated, especially when upsampling.

2. Bilinear Interpolation

- A common interpolation technique used for images
 - A smoother image than the nearest neighbor interpolation.
- The newly generated pixel is a weighted sum of the nearest pixels.
- The weights are determined linearly; each weight is directly proportional to the distance from each existing pixel



Bilinear Interpolation

- ▶ Bilinear interpolation considers the closest 2x2 neighborhood of known pixels surrounding the unknown pixel. The value of the unknown pixel is a weighted average of these 4 pixels, based on their distance.

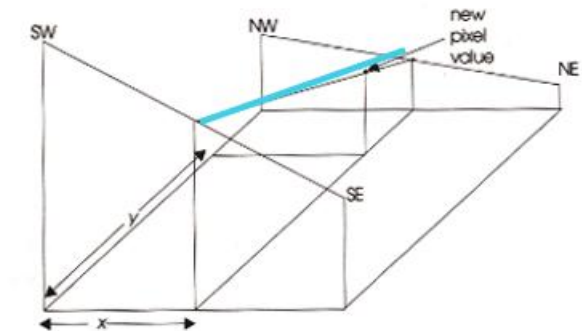
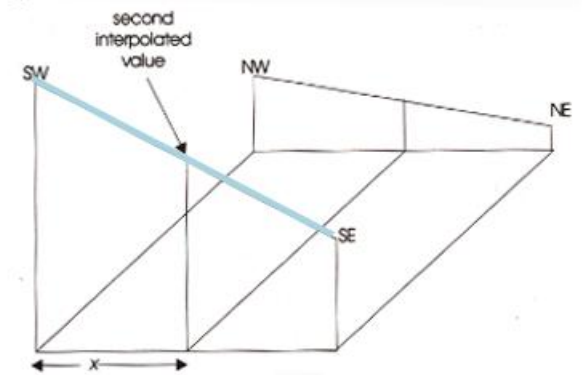
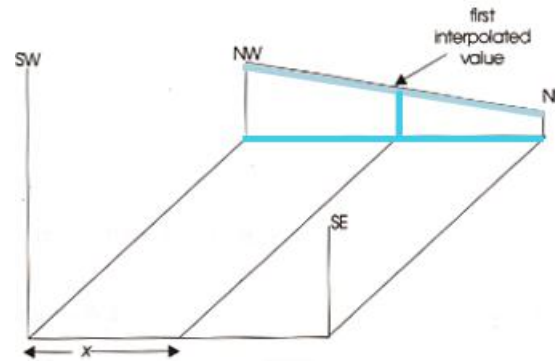
Process:

Calculate the horizontal interpolation between two pixels (left and right).
Then, interpolate vertically between the two resulting values.

- **Pros:**
Produces smoother results compared to nearest neighbor.
- **Cons:**
More computationally expensive than nearest neighbor.

Bilinear Interpolation

➤ Steps →



➤ code

```
EWweight=floatx-floor(x);
```

```
NSweight=floaty-floor(y);
```

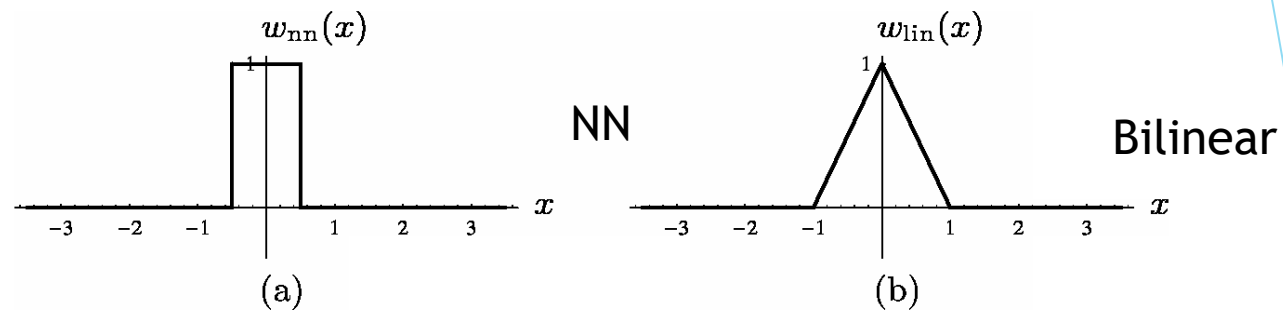
```
EWtop=NW+EWweight*(NE-NW);
```

```
Ewbottom=SW+EWweight*(SE-SW);
```

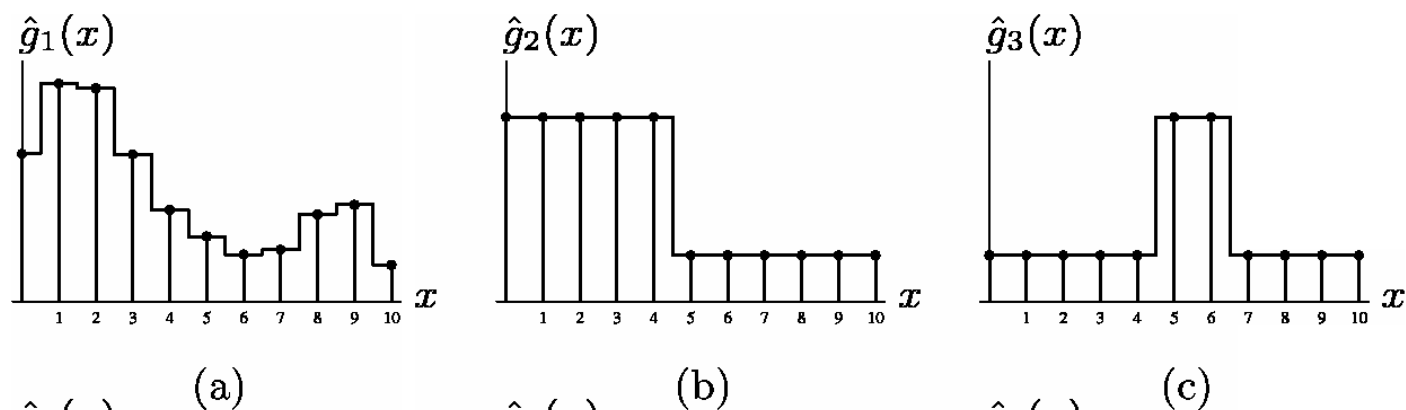
```
Dest_pixel=(char)EWtop+NSweight*(Ewbottom-EWtop)
```

Geometric transformations

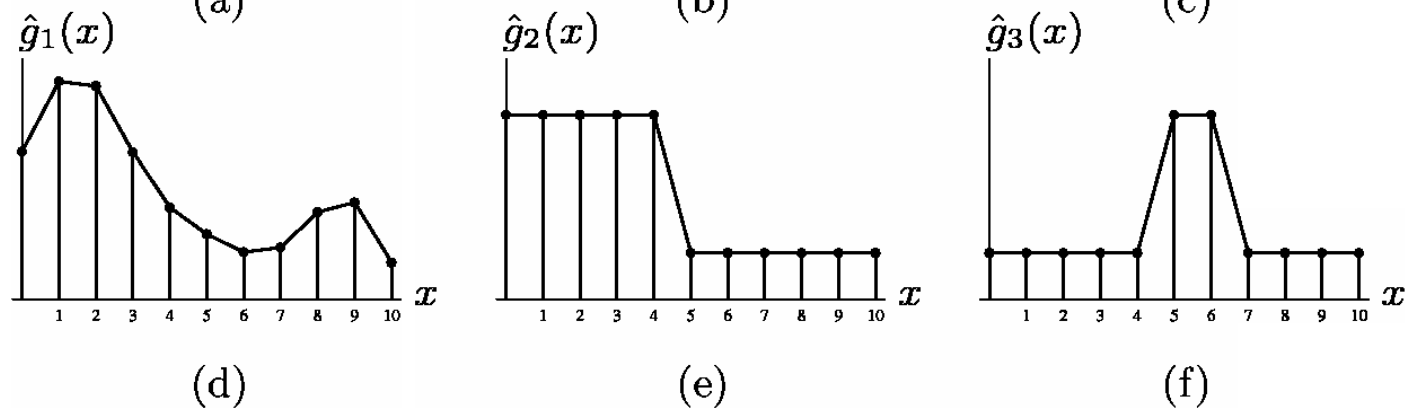
Interpolation kernels:



Nearest Neighbor

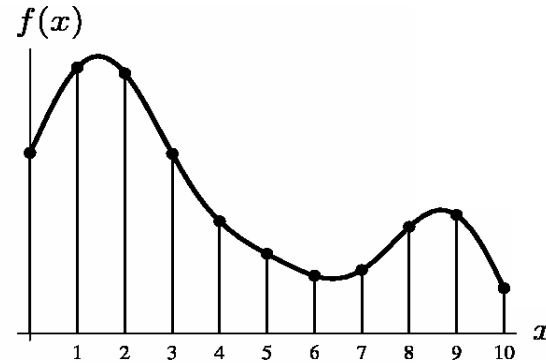
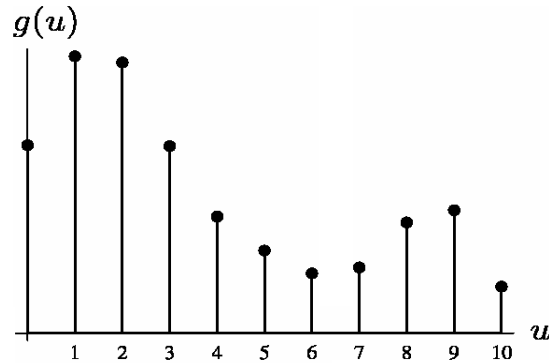


Linear

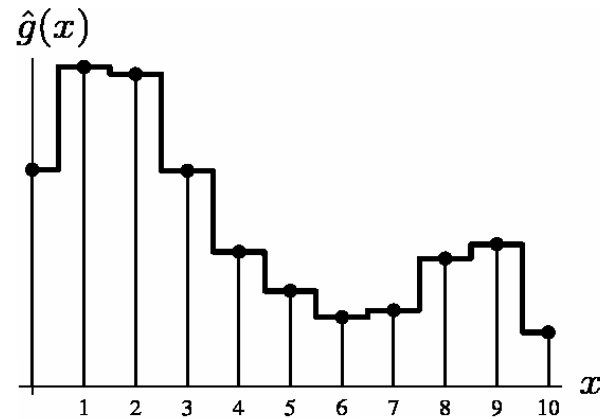


Geometric transformations

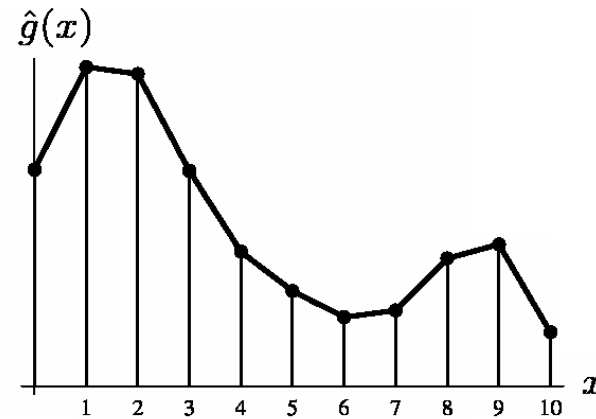
How to interpolate?



Nearest
neighbor



(a)



(b)

Bilinear

$$\hat{g}(x) = g(u_0),$$

where $u_0 = \text{round}(x) = \lfloor x + 0.5 \rfloor$

$$\begin{aligned}\hat{g}(x) &= g(u_0) + (x - u_0) \cdot (g(u_0 + 1) - g(u_0)) \\ &= g(u_0) \cdot (1 - (x - u_0)) + g(u_0 + 1) \cdot (x - u_0)\end{aligned}$$

Geometric transformations

Generalization of Interpolation:

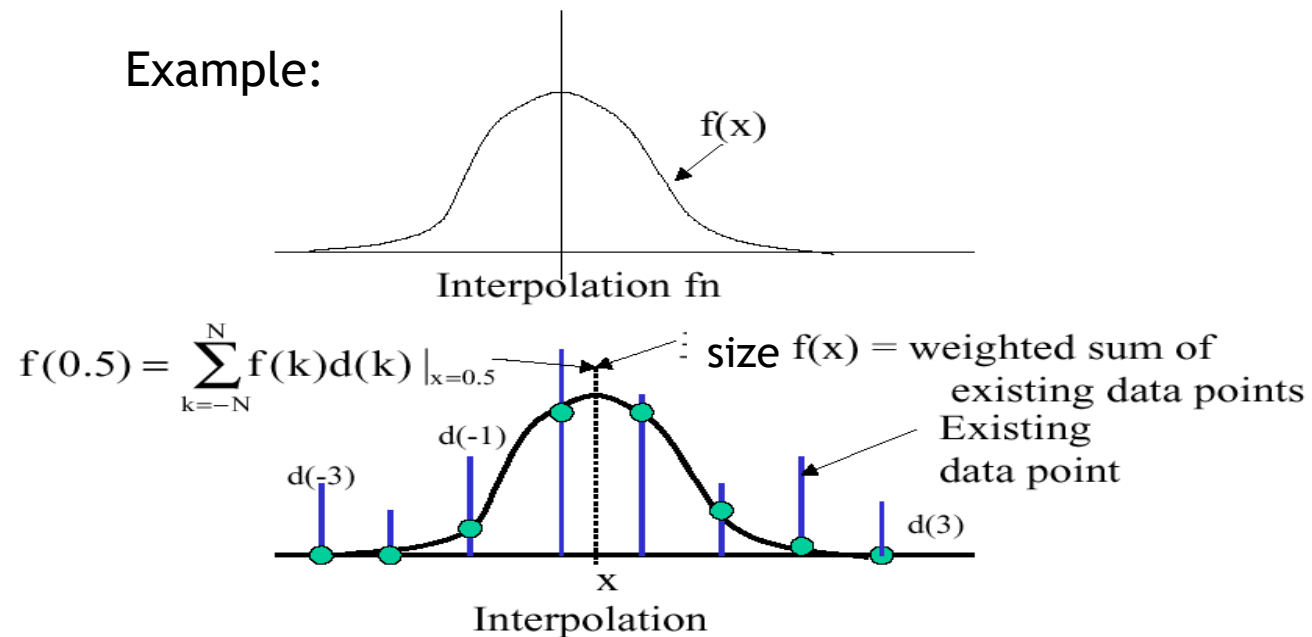
In general, interpolation is implemented by convolution of discrete function $g(u)$ with some continuous interpolation kernel $w(x)$

$$\hat{g}(x_0) = [w * g](x_0) = \sum_{u=-\infty}^{\infty} w(x_0 - u) \cdot g(u)$$

Higher order interpolation

- ▶ 1-dimension interpolation
 - ▶ A function is centered on the single point of interest.
 - ▶ The values of function at the sample points are multiplied by those samples.
 - ▶ The sum of the products is the value of the newly generated pixel.

Example:



Higher order interpolation

- ▶ Higher order interpolation
 - ▶ Cubic convolution and B-Spline interpolation functions
 - ▶ Requires large areas to generate a new pixel => 16 nearest pixels each
 - ▶ Cf. Nearest neighbor requires 1 pixel, and Bilinear 4 pixels.
 - ▶ Principle of interpolation function processes
 - ▶ A function is centered on the single point of interest.
 - ▶ The values of function at the sample points are multiplied by those samples.
 - ▶ Sample points: $f(-d)$, $f(-1-d)$, $f(1-d)$, $f(2-d)$, where x is offset from the nearest left sample by d .
 - ▶ The sum of the products is the value of the newly generated pixel.

Cubic Convolution Interpolation

- The 1-D cubic convolution function : Fig. 4.7

$$f(x) = \begin{cases} (a+2)|x|^3 - (a+3)|x|^2 + 1 & 0 \leq |x| < 1 \\ a|x|^3 - 5a|x|^2 + 8a|x| - 4a & 1 \leq |x| < 2 \\ 0 & 2 \leq |x| \end{cases}$$

- The output is clipped when the output is
 - negative pixel value
 - Exceeding maximum pixel values

Cubic Convolution Interpolation



256×256
original
image



128×125 original image



Cubic interpolation
256×256 scale up image

3. Bicubic Interpolation

- ▶ Bicubic interpolation goes one step further than bilinear interpolation by considering the closest 4x4 neighborhood of known pixels. It calculates the weighted average of these 16 pixels, offering even smoother results.
- **Process:**
 - Use cubic polynomials to interpolate along the x-axis and y-axis, using a 4x4 grid of surrounding pixels.

Bicubic Interpolation

Mathematical Formula: Bicubic interpolation uses a cubic polynomial function to determine the new pixel value. The general form for cubic interpolation in one dimension is:

$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3$$

The exact calculation requires solving for the coefficients a_0, a_1, a_2, a_3 based on the neighboring pixels.

- Advantage:**

Produces much smoother and visually appealing results than bilinear interpolation.

- Cons:**

Significantly more computationally expensive.

Requires more memory to store the surrounding pixels.

4. Spline Interpolation

- ▶ Spline interpolation fits a piecewise polynomial to the data points, usually using cubic splines. It is commonly used for more precise image transformations and can provide smoother results than bicubic interpolation, especially in cases where very high-quality results are needed.
- **Process:**
 - Fit a cubic spline (a smooth, continuous curve) to the data points and interpolate values using that curve.
- **Pros:**
 - Can provide smoother results with less visible artifacts.
- **Cons:**
 - Computationally intensive and more complex to implement.

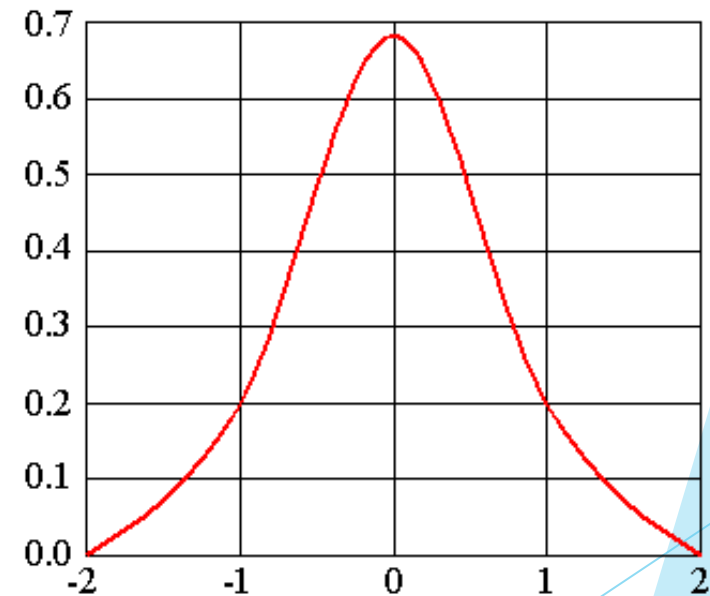
B-Spline Interpolation

✚ B-Spline Interpolation

- Yields the greatest smoothing among all interpolation function
- The B-spline makes a reasonably good low-pass filter.
 - The ideal interpolation function is a low pass filter
- No clipping is needed.

✚ The 1-D B-Spline function

$$f(x) = \begin{cases} 1/2 |x|^3 - |x|^2 + 2/3 & 0 \leq |x| < 1 \\ -1/6 |x|^3 + |x|^2 - 2|x| + 3/4 & 1 \leq |x| < 2 \\ 0 & 2 \leq |x| \end{cases}$$



B-spline interpolation function

B-Spline Interpolation



Original



Cubic B-spline

Practice code

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load the image
image = cv2.imread('pepper.png')

# Resize the image using different interpolation methods
image_nn = cv2.resize(image, (600, 600), interpolation=cv2.INTER_NEAREST) # Nearest Neighbor
image_bilinear = cv2.resize(image, (600, 600), interpolation=cv2.INTER_LINEAR) # Bilinear
image_bicubic = cv2.resize(image, (600, 600), interpolation=cv2.INTER_CUBIC) # Bicubic

# Plot the results
plt.figure(figsize=(10, 10))

plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title('Original Image')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(cv2.cvtColor(image_nn, cv2.COLOR_BGR2RGB))
plt.title('Nearest Neighbor')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(image_bilinear, cv2.COLOR_BGR2RGB))
plt.title('Bilinear Interpolation')
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(image_bicubic, cv2.COLOR_BGR2RGB))
plt.title('Bicubic Interpolation')
plt.axis('off')

plt.show()
```

Practice code output

Original Image



Nearest Neighbor



Bilinear Interpolation



Bicubic Interpolation

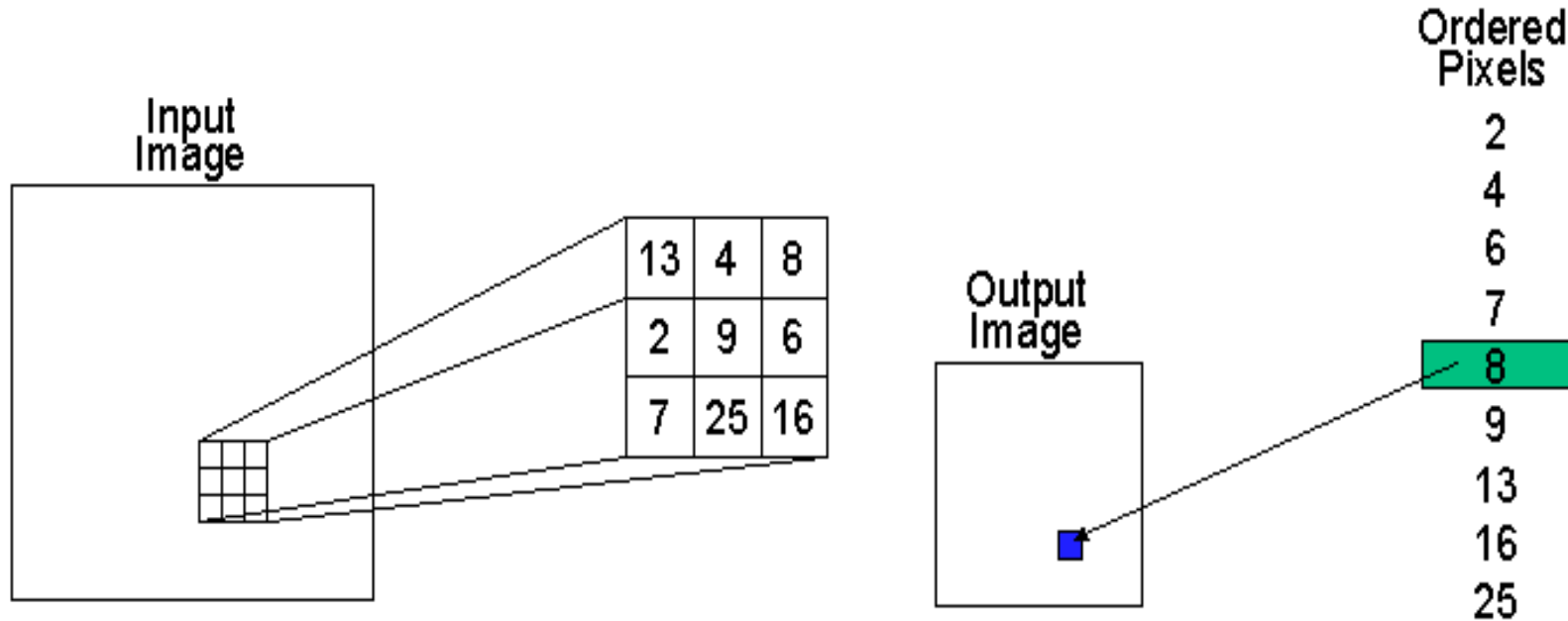


Image Scaling

- ✚ Enlarging image: Magnification, scaling up, zooming, up-sampling, stretching.
- ✚ Reducing image: Shrinking, scaling down, decimation, down-sampling, minification
- ◆ Two important things
 1. Never improve upon the original image resolution
 2. The resulting image will degrade with every operation

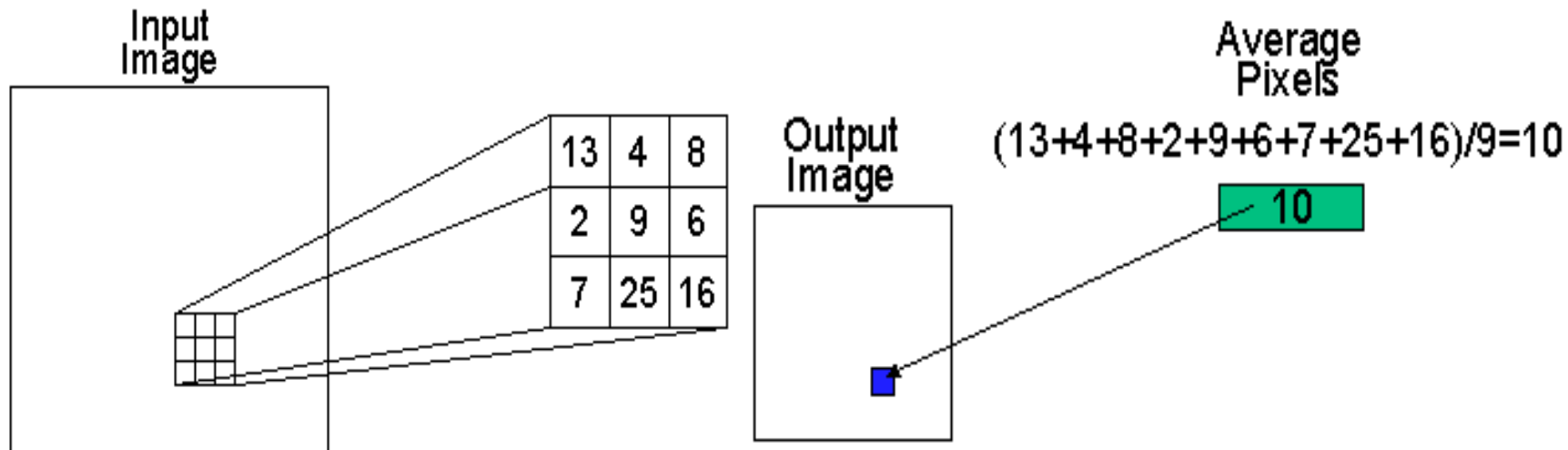
Minification techniques

- ✚ Median representation
 - ✚ Basic concept : represent a block of pixels with one pixel
 - ◆ $n \times n$ window
 - ◆ Replace a block of pixels with its median value

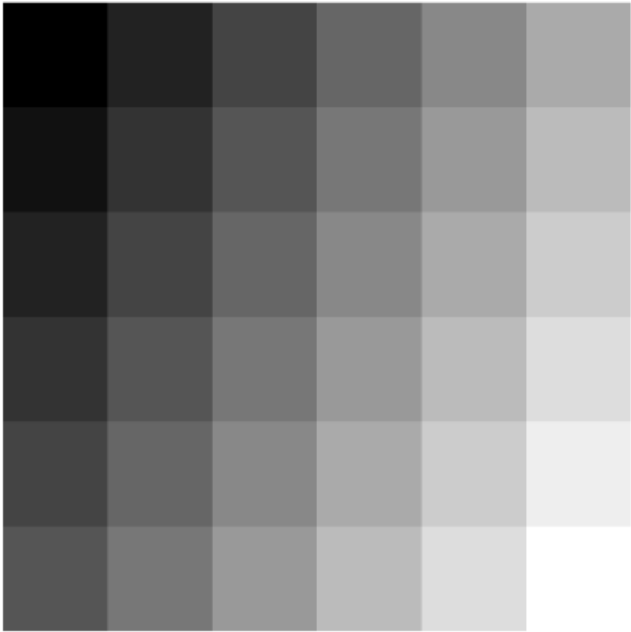


Minification techniques

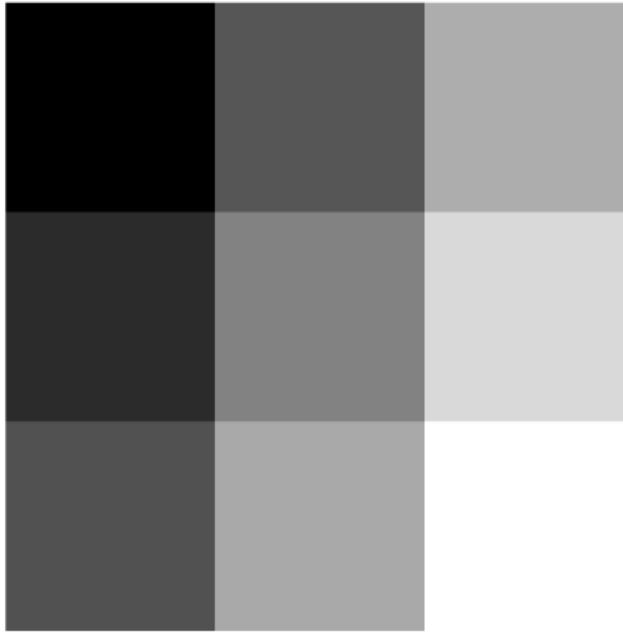
- ✚ Average representation
 - ✚ n x n window
 - ✚ Represent a block of pixels with an average of all of pixels.



Minification techniques



Original image 6x6



minified image 3x3
using bilinear interpolation

Practice code

```
from PIL import Image
import numpy as np

# Example 6x6 grayscale image (using a NumPy array)
image_data = np.array([
    [10, 20, 30, 40, 50, 60],
    [15, 25, 35, 45, 55, 65],
    [20, 30, 40, 50, 60, 70],
    [25, 35, 45, 55, 65, 75],
    [30, 40, 50, 60, 70, 80],
    [35, 45, 55, 65, 75, 85]
])

# Convert the NumPy array into an image object
image = Image.fromarray(image_data.astype('uint8'))

# Resize the image to 3x3 (minification) using bilinear interpolation
minified_image = image.resize((3, 3), Image.BILINEAR)

# Show the original and the minified image
image.show(title="Original 6x6 Image")
minified_image.show(title="Minified 3x3 Image")

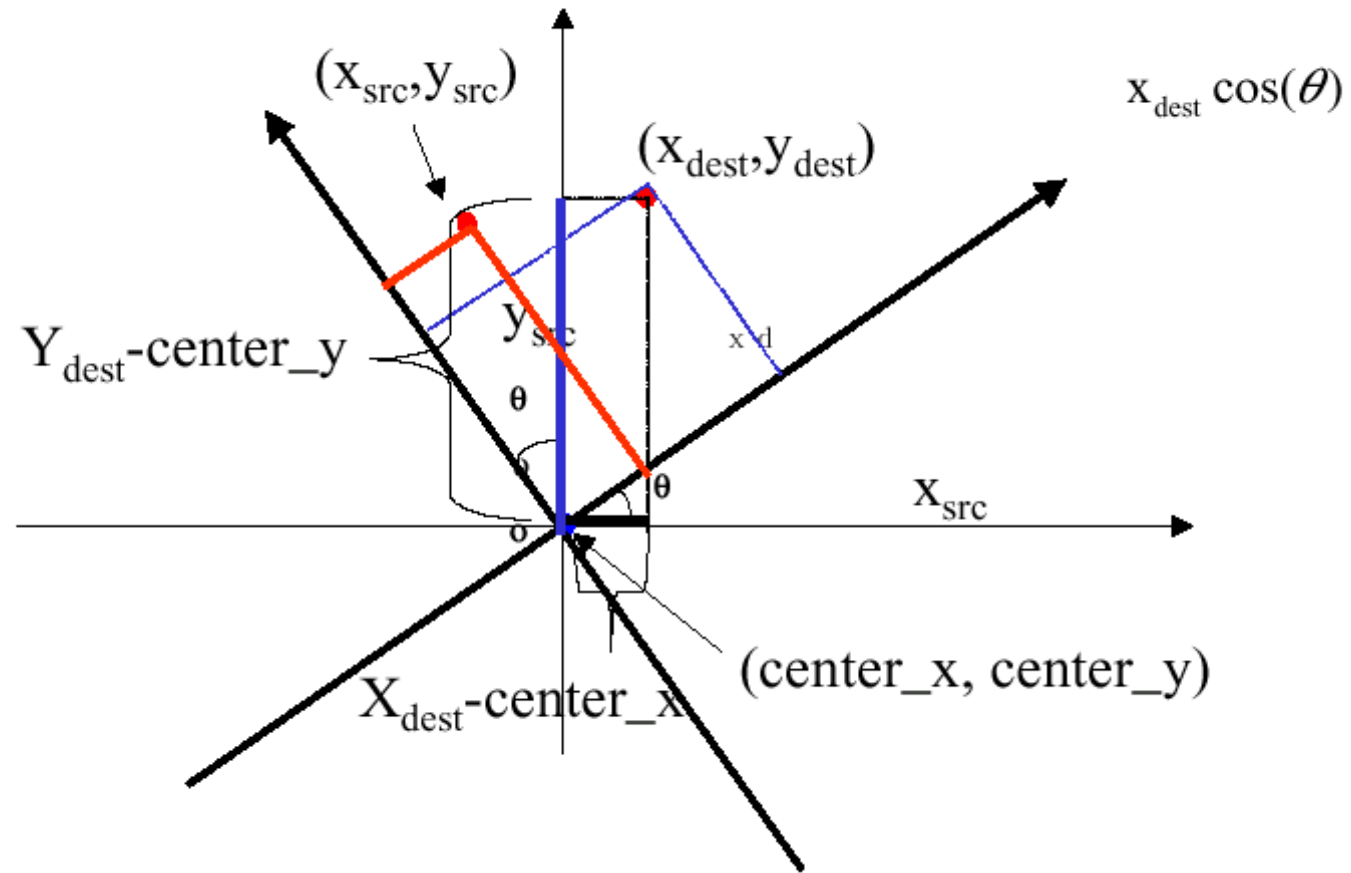
# Convert the minified image back to a NumPy array to see the pixel values
minified_image_data = np.array(minified_image)
print("Minified Image Data (3x3):")
print(minified_image_data)
```

Minified Image
Data (3x3):
[[21 39 57]
[30 48 66]
[38 56 74]]

Rotation

- ◆ Rotates an image about its center pixel through any given angle.
- ◆ Rotation about point (0,0) (Inverse transformation)
$$X_Source = X_Dest * \cos(\text{angle}) + Y_Dest * \sin(\text{angle})$$
$$Y_Source = Y_Dest * \cos(\text{angle}) - X_Dest * \sin(\text{angle})$$
- ◆ Rotation about image center:
$$X_source = (X_dest - \text{center_x}) * \cos(\text{angle}) + (-1 * (Y_dest - \text{center_y}) * \sin(\text{angle}) + 128)$$
$$Y_source = (Y_dest - \text{center_y}) * \cos(\text{angle}) + (X_dest - \text{center_x}) * \sin(\text{angle}) + 128$$
- ◆ Rotation by multiples of 90°: a simple transposition of rows and columns in an image.

Rotation



Rotation



original

Angle 45 : rotation

Angle 90 : rotation

Practice code

```
from PIL import Image

# Open an image file
image = Image.open('pepper.png')

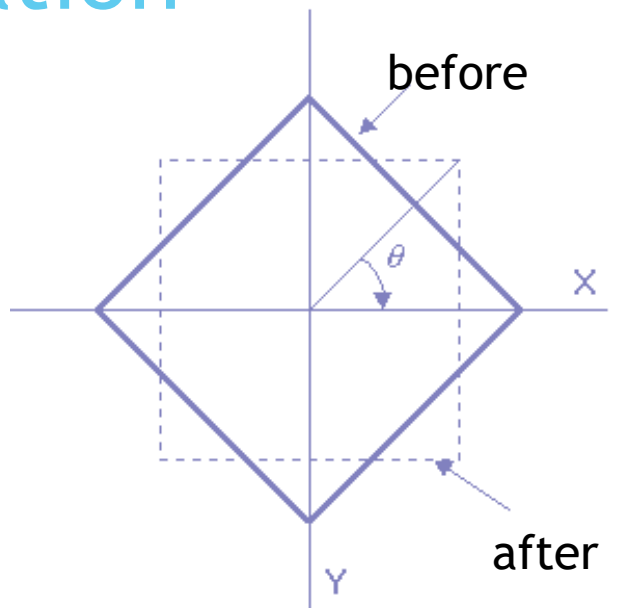
# Rotate the image by 45 degrees
rotated_image = image.rotate(45, expand=True) # 'expand=True' prevents cropping

# Save the rotated image
rotated_image.save('rotated_image.png')

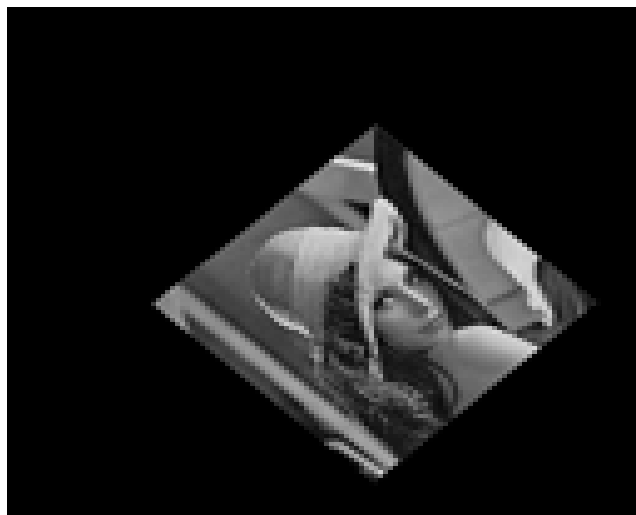
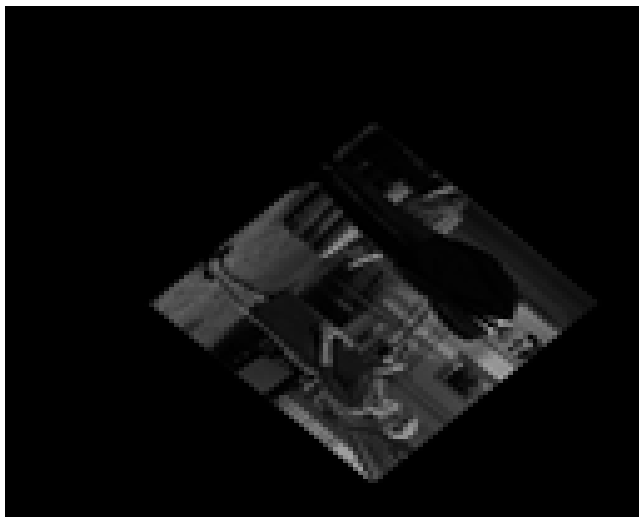
# Show the original and rotated images
image.show(title="Original Image")
rotated_image.show(title="Rotated Image")
```



Rotation



Rotation and scaling



Translation

- ◆ Move a section of the image to another location in the image.
- ◆ Double buffering for partial translation

(X, Y, Z) and displacement (X_0, Y_0, Z_0)

$$\begin{aligned} X^* &= X + X_0 \\ Y^* &= Y + Y_0 \\ Z^* &= Z + Z_0 \end{aligned} \Rightarrow \begin{bmatrix} X^* \\ Y^* \\ Z^* \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & X_0 \\ 0 & 1 & 0 & Y_0 \\ 0 & 0 & 1 & Z_0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

or

$$\begin{bmatrix} X^* \\ Y^* \\ Z^* \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & X_0 \\ 0 & 1 & 0 & Y_0 \\ 0 & 0 & 1 & Z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\Rightarrow \underline{v}^* = T \underline{v}$$

Translation



Original

X 30, y 45 translation

Practice code

```
from PIL import Image, ImageChops

# Open an image file
image = Image.open('pepper.png')

# Define translation offsets (x and y)
x_offset = 100 # Shift 100 pixels to the right
y_offset = 50  # Shift 50 pixels down

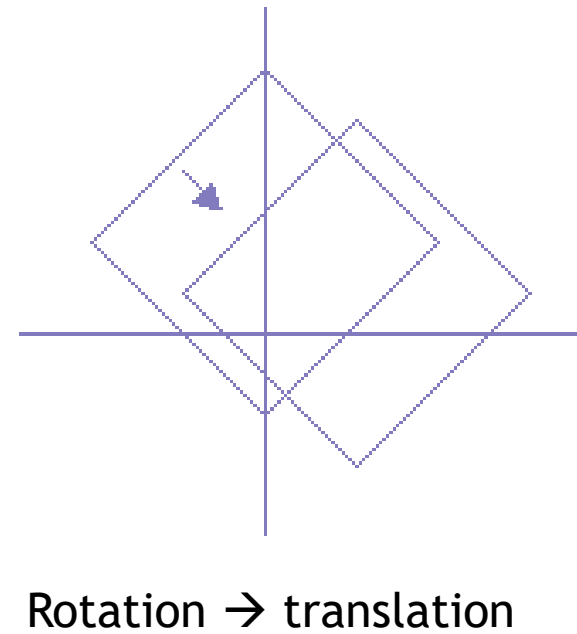
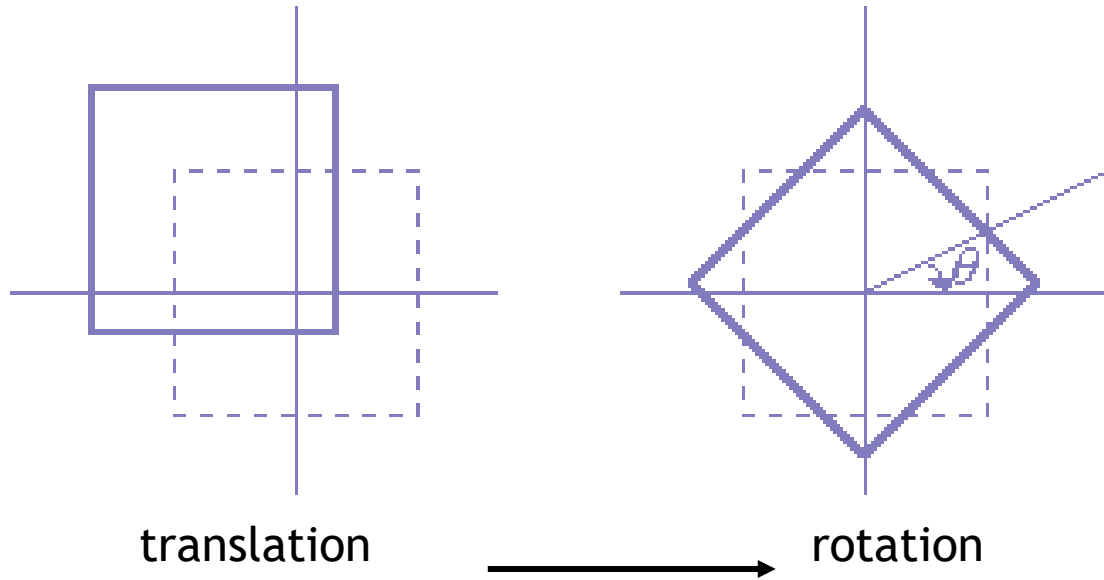
# Create a new blank image with the same size and mode as the original
translated_image = ImageChops.offset(image, x_offset, y_offset)

# Save the translated image
translated_image.save('translated_image.png')

# Show the original and translated images
image.show(title="Original Image")
translated_image.show(title="Translated Image")
```



Translation & rotation



Translation



Mirroring

- ◆ Flip the image about x or y axis.
- ◆ Horizontal mirroring & Vertical mirroring
- ◆ Pure geometric process

Mirroring



original

Horizontal mirroring

Vertical mirroring

Mirroring

- ▶ In horizontal mirroring, the image is flipped along the **vertical axis** (Y-axis). For every pixel at position (x, y), its new position becomes
- ▶ (W - 1 - x, y) where W is the width of the image.

$$\begin{bmatrix} A & B & C \\ D & E & F \\ G & H & I \end{bmatrix}$$

$$\begin{bmatrix} C & B & A \\ F & E & D \\ I & H & G \end{bmatrix}$$

After **horizontal mirroring**, the image becomes:

Practice code

```
from PIL import Image

# Open an image file
image = Image.open('pepper.png')

# Horizontal mirror (flip left to right)
mirrored_horizontal = image.transpose(Image.FLIP_LEFT_RIGHT)

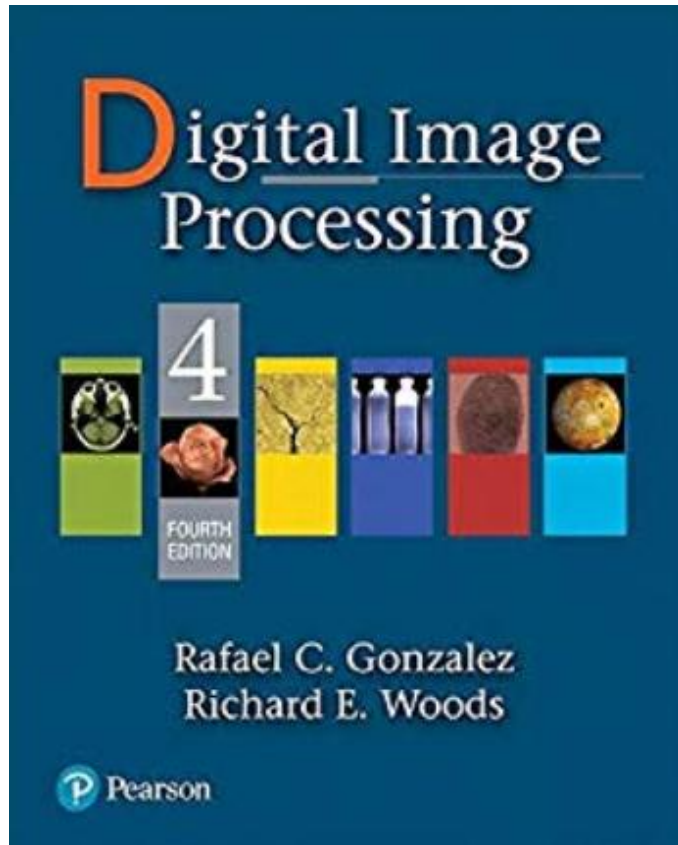
# Vertical mirror (flip top to bottom)
mirrored_vertical = image.transpose(Image.FLIP_TOP_BOTTOM)

# Save the mirrored images
mirrored_horizontal.save('mirrored_horizontal.png')
mirrored_vertical.save('mirrored_vertical.png')

# Show the mirrored images
mirrored_horizontal.show(title="Horizontally Mirrored Image")
mirrored_vertical.show(title="Vertically Mirrored Image")
```



Reference



Chapter - 4

Question



Thank you

